

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ОМСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
Операционные системы

Отчёт
по лабораторной работе №9

Выполнил:

Студент группы ПИ-221

Нурсалимов Данияр Ерланович

Проверил(а): _____

Дата сдачи отчета: _____

Задание. Разработать программы в ОС Windows для двух отдельных процессов, использующих общую память. В первой программе должна создаваться разделяемая память и семафор для взаимодействия между процессами. Во второй открывается доступ к этой же разделяемой памяти и семафору. Первая программа должна после задержки в 10 - 15 секунд записать какой-то текст в разделяемую память и указать его готовность с помощью семафора, а вторая программа должна вначале прочитать текстовую информацию из того места разделяемой памяти, где она должна появиться и выдать полученную информацию на экран с примечанием о существе действия, затем перейти к ожиданию разрешающего значения семафора и только после его завершения выдать на экран содержимое из разделяемой памяти с соответствующим примечанием. Вместо семафора в Windows можно использовать событие или мьютекс. Обе программы должны после получения действующего указателя на область общей памяти вывести адрес этой области в консольное окно с соответствующими пояснениями. Первая программа, кроме того, после формирования данных для второго процесса, должна запросить 1000 байтов дополнительной памяти с помощью универсальной функции VirtualAlloc и записать в эту область памяти, начиная с ее начала, сколько возможно символов латинского алфавита, записывая эти символы, пропуская по 399 свободных байтов (т.е. записывая 'a' в байт с нулевым смещением этой области памяти, затем 'b' в байт со смещением 400, далее 'c' в байт со смещением 800 и т.д., выполняя такое продвижение на 400 байтов по крайней мере 15 раз). Каждую такую запись байта сопровождать детальным сообщением о действии и числовым значением адреса, по которому размещается этот байт в памяти. Наблюдаемые результаты объяснить. (Базовый вариант)

Исходный код базовой программы на Windows:

lab9.c:

```
#define BUF_SIZE 15000 // Определение размера буфера  
#include <Windows.h>
```

```

#include <stdio.h>
#include <conio.h>
#include <tchar.h>

// Определение строковых констант
TCHAR szName[] = _T("Main");
TCHAR fMsg[] = _T("Hello from Main");
TCHAR szMsg[] = _T("Message from the first process");
LPCSTR Semaphore = "Semaphore";

// Определение текстовой строки
char AllocText[] = "abcdefghijklmno";

int main()
{
    system("cls"); // очистка консоли
    SetConsoleOutputCP(1251); // установка формата вывода
    SetConsoleCP(1251); // установка формата ввода
    char buff[255]; // создание буфера для вывода сообщений
    int len;

    HANDLE Console = GetStdHandle(STD_ERROR_HANDLE); //
получение дескриптора консоли
    len = sprintf(buff, "Main program\n"); // формирование сообщения для
вывода
    WriteFile(Console, buff, len, NULL, NULL); // вывод сообщения в
консоль
    HANDLE hSemaphore = CreateSemaphoreA(NULL, 0, 1, Semaphore); //
создание семафора
    HANDLE hMapFile =
CreateFileMapping(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE,
0, BUF_SIZE, szName); // создание файла-отображения памяти

```

```

        LPTSTR pBuf = (LPTSTR)MapViewOfFile(hMapFile,
FILE_MAP_ALL_ACCESS, 0, 0, 0); // отображение файла-отображения
памяти в адресное пространство процесса

        len = sprintf(buff, "Writing the first message to memory");
        WriteFile(Console, buff, len, NULL, NULL);

        CopyMemory((PVOID)pBuf, fMsg, (_tcslen(fMsg) * sizeof(TCHAR)));
// копирование сообщения в отображенную память

        Sleep(2000); // ожидание 2 секунды

        STARTUPINFO sInfo; // структура для запуска нового процесса

        PROCESS_INFORMATION pInfo; // структура для хранения
информации о запущенном процессе

        ZeroMemory(&sInfo, sizeof(STARTUPINFO)); // обнуление структуры
sInfo.cb = sizeof(sInfo);

        TCHAR path[MAX_PATH] = _T("app.exe"); // путь к запускаемому
файлу

        if (CreateProcess(NULL, path, NULL, NULL, FALSE,
CREATE_NEW_CONSOLE, NULL, NULL, &sInfo, &pInfo) == 0) { // запуск
программы и проверка на ошибки

            len = sprintf(buff, "File app.exe not found\n");
            WriteFile(Console, buff, len, NULL, NULL);
        }

        len = sprintf(buff, "Waiting for 10 seconds\n");
        WriteFile(Console, buff, len, NULL, NULL);
        Sleep(10000);

        len = sprintf(buff, "Writing:");
        WriteFile(Console, buff, len, NULL, NULL);

        len = sprintf(buff, "(messages from the first process to the program)\n");
        WriteFile(Console, buff, len, NULL, NULL);

        CopyMemory((PVOID)pBuf, szMsg, (_tcslen(szMsg) *
sizeof(TCHAR))); // копирование сообщения в отображенную память

```

```
ReleaseSemaphore(hSemaphore, 1, NULL); // освобождение семафора  
LPTSTR NewMem = (LPTSTR)VirtualAlloc(NULL, BUF_SIZE,  
MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE); // выделение  
виртуальной памяти
```

```
if (NewMem != NULL) {  
    len = sprintf(buff, "Virtual memory created\n");  
    WriteFile(Console, buff, len, NULL, NULL);  
}  
else {  
    len = sprintf(buff, "Failed\n");  
    WriteFile(Console, buff, len, NULL, NULL);  
    _getch();  
    return 1;  
}
```

```
LPTSTR Address = NewMem; // указатель на начало выделенной  
виртуальной памяти
```

```
for (int i = 0; i < 15; i++) {  
    CopyMemory((LPVOID)Address, (PVOID)AllocText, sizeof(char)); //  
копирование текста в выделенную виртуальную память
```

```
    len = sprintf(buff, "Address:");  
    WriteFile(Console, buff, len, NULL, NULL);  
    len = sprintf(buff, "0x%X", Address);  
    WriteFile(Console, buff, len, NULL, NULL);  
    printf("(0x%X) ", Address);  
    len = sprintf(buff, "Value:");  
    WriteFile(Console, buff, len, NULL, NULL);  
    printf("Value: ");  
    len = sprintf(buff, "(%c)\n", AllocText[i]);  
    WriteFile(Console, buff, len, NULL, NULL);  
    Address += 400; // сдвиг указателя на 400 байт
```

```

        Sleep(200); // ожидание 200 миллисекунд
    }
    len = sprintf(buff, "\n\t Press anything to exit the program");
    WriteFile(Console, buff, len, NULL, NULL);
    getchar(); // ожидание нажатия клавиши
    return 0;
}

```

lab9-2.c:

```

#define BUF_SIZE 1024
#include <Windows.h>
#include <stdio.h>
#include <conio.h>
#include <tchar.h>

// Определение строковых констант с помощью макроса TEXT()
TCHAR szName[] = TEXT("Main");
TCHAR vaName[] = TEXT("VMAlloc");
LPCWSTR Semaphore = L"Semaphore";

int main() {
    // Установка кодировки консоли на кириллицу
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);
    char buff[255];
    int len;
    HANDLE hMapFile;
    LPCTSTR pBuf;

    // Открытие существующего семафора по его имени
    HANDLE hSemaphore =
OpenSemaphoreW(SEMAPHORE_ALL_ACCESS, 0, Semaphore);

```

```

HANDLE Console = GetStdHandle(STD_ERROR_HANDLE);
len = sprintf(buff, "app.exe \n");// Вывод сообщения в консоль
WriteFile(Console, buff, len, NULL, NULL);
// Проверка успешности открытия семафора
if (hSemaphore != NULL) {
    len = sprintf(buff, "Semaphore opened successfully\n");
    WriteFile(Console, buff, len, NULL, NULL);
}
else {
    len = sprintf(buff, "Error: %d\n", GetLastError());
    WriteFile(Console, buff, len, NULL, NULL);
    _getch();
    return 1;
}
// Открытие файла-отображения в памяти
hMapFile = OpenFileMapping(FILE_MAP_ALL_ACCESS, FALSE,
szName);

pBuf = (LPTSTR)MapViewOfFile(hMapFile,
FILE_MAP_ALL_ACCESS, 0, 0, BUF_SIZE);// Создание указателя на
отображенный в памяти участок

MessageBox(NULL, pBuf, TEXT("Process 2 - first message"),
MB_OK);// Вывод сообщения в диалоговое окно

WaitForSingleObject(hSemaphore, INFINITE);// Ожидание сигнала от
семафора

MessageBox(NULL, pBuf, TEXT("Process 2 - second message"),
MB_OK);// Вывод сообщения в диалоговое окно

len = sprintf(buff, "\n\t Press ESC to exit the program");// Вывод
сообщения в консоль

WriteFile(Console, buff, len, NULL, NULL);
// Бесконечный цикл ожидания нажатия клавиши ESC

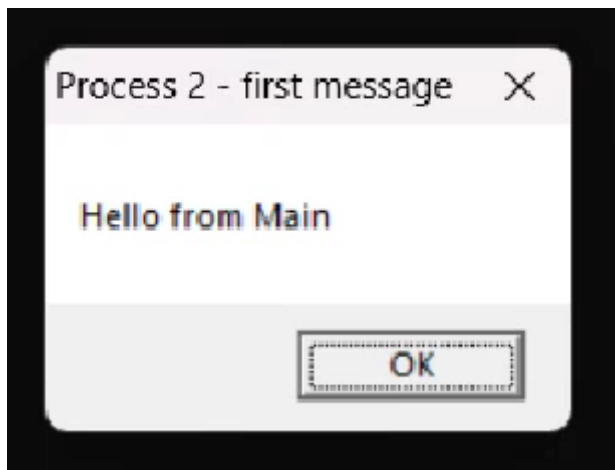
```

```
while (1) {  
    if (GetAsyncKeyState(27)) { // Освобождение ресурсов и выход из  
программы  
        UnmapViewOfFile(pBuf);  
        CloseHandle(hMapFile);  
        CloseHandle(hSemaphore);  
        break;  
    }  
}  
return 0;  
}
```

cmndfile.bat:

```
gcc -o lab9.exe lab9.c  
gcc -o lab9-2.exe lab9-2.c  
start lab9.exe  
start lab9-2.exe
```

Результаты базового задания:



```
C:\OmGTU\third term\OS\lab  X + v
Main program
Writing the first message to memoryFile app.exe not found
Waiting for 10 seconds
Writing:(messages from the first process to the program)
Virtual memory created
Address:0xB4960000(0xB4960000) Value:Value: (a)
Address:0xB4960190(0xB4960190) Value:Value: (b)
Address:0xB4960320(0xB4960320) Value:Value: (c)
Address:0xB49604B0(0xB49604B0) Value:Value: (d)
Address:0xB4960640(0xB4960640) Value:Value: (e)
Address:0xB49607D0(0xB49607D0) Value:Value: (f)
Address:0xB4960960(0xB4960960) Value:Value: (g)
Address:0xB4960AF0(0xB4960AF0) Value:Value: (h)
Address:0xB4960C80(0xB4960C80) Value:Value: (i)
Address:0xB4960E10(0xB4960E10) Value:Value: (j)
Address:0xB4960FA0(0xB4960FA0) Value:Value: (k)
Address:0xB4961130(0xB4961130) Value:Value: (l)
Address:0xB49612C0(0xB49612C0) Value:Value: (m)
Address:0xB4961450(0xB4961450) Value:Value: (n)
Address:0xB49615E0(0xB49615E0) Value:Value: (o)

Press anything to exit the program|

C:\OmGTU\third term\OS\lab  X +
app.exe
Semaphore opened successfully
```

